

Technical Architecture and Integration

Analysis of the Better-T-Stack: A Professional Ecosystem Report

The modern landscape of full-stack development has shifted from monolithic structures toward modular, type-safe ecosystems that prioritize developer experience without compromising on production-grade performance. The Better-T-Stack represents a sophisticated convergence of these ideals, particularly when configured with a Next.js and Native-Uniwind frontend, an Express backend running on Node.js, and a persistence layer powered by Turso and Drizzle ORM.¹ This architectural analysis explores the intricacies of a specific deployment configuration, examining the mechanisms of type-safe API communication through tRPC, the complexities of multi-runtime authentication via Better-Auth, and the economic advantages of the Polar Merchant of Record model for global monetization.³

Scaffolding Logic and CLI Implementation

The foundation of the project begins with the create-better-t-stack CLI, a tool designed to minimize the repetitive overhead of initializing monorepo projects. By executing the command `pnpm create better-t-stack@latest NAME_HERE` with the specified flags, the generator initializes a Turborepo-managed workspace that integrates diverse technologies into a single source of truth.¹ The philosophy underpinning this CLI is the "Roll Your Own" approach, where only the requested components are included, ensuring zero bloat in the final scaffold.²

A critical aspect of the CLI's operation is its telemetry and extension system. The tool collects anonymous usage data, such as configuration choices and OS platform, to improve future versions, though this can be disabled globally via the `BTS_TELEMETRY_DISABLED` environment variable.¹ Furthermore, the CLI supports advanced agentic workflows through the `add-mcp` addon, allowing developers to install the stack into supported AI agent configurations without relying on global installs.¹

Configuration Flag	Selection	Architectural Significance
--frontend	next, native-unwind	Establishes a universal UI strategy for web and mobile.

--backend	express	Provides a robust, standalone Node.js server environment.
--api	trpc	Ensures end-to-end type safety between server and clients.
--auth	better-auth	Implements a framework-agnostic session management layer.
--payments	polar	Offloads global tax compliance and MoR duties.
--database	sqlite	Utilizes libSQL for edge-compatible persistence.
--orm	drizzle	Provides a lightweight, TypeScript-first SQL wrapper.
--db-setup	turso	Enables distributed SQLite replication.
--addons	pwa, turborepo	Adds web-installability and build-system orchestration.

Monorepo Orchestration and Build Systems

The choice of pnpm as the package manager and turborepo as the build orchestrator is strategic for managing a stack that spans both web and native platforms. pnpm utilizes a

content-addressable store to save disk space and a symlink-based node_modules structure to prevent phantom dependencies, which is vital in a monorepo where multiple apps share local packages.⁶ Turborepo complements this by creating a dependency-aware execution pipeline, allowing for concurrent task execution and remote caching of build artifacts.⁶

The project structure typically allocates applications into an apps/ directory and shared logic into a packages/ directory. This separation allows the Next.js frontend, the Express backend, and the React Native application to consume shared configurations for TypeScript, linting, and database schemas.⁵

Workspace Directory Structure

The resulting monorepo architecture organizes the configuration into functional domains, ensuring that a change in the shared API or database package propagates type errors throughout the entire stack during development.⁷

Directory	Purpose	Key Technologies
apps/web	Primary web application	Next.js, Tailwind v4
apps/native	Cross-platform mobile app	React Native, Expo, Unwind
apps/server	Backend API host	Express, Node.js
packages/api	API contracts and routers	tRPC v11, Zod
packages/db	Data persistence layer	Drizzle ORM, libSQL
packages/ui	Shared design system	Tailwind v4, shadcn/ui
packages/auth	Unified authentication logic	Better-Auth

Frontend Architecture: Next.js and Native-Uniwind

The dual-frontend configuration (Next.js and React Native) addresses the need for a high-performance web presence alongside a native mobile experience. Next.js, utilizing the App Router and React Server Components (RSC), serves as the primary web gateway. It handles server-side rendering (SSR) and SEO-optimized content while proxying business logic to the standalone Express server.⁹

Native-Uniwind and Universal Styling

The mobile component utilizes native-uniwind, a template that leverages Tailwind CSS v4 to provide a shared styling language between web and native.¹¹ Unlike traditional React Native styling, which relies on `StyleSheet.create`, Uniwind allows developers to use utility classes that the NativeWind compiler translates into native styling objects.¹¹ This configuration uses Expo SDK 54 and React Native 0.81, providing a modern foundation for mobile development.¹¹

A significant challenge identified in this setup involves relative path resolution for styling libraries in monorepos. Specifically, the import path for HeroUI styles in the Expo/Uniwind template may require manual adjustment. The default path in `global.css` often points to `./node_modules/heroui-native/lib`, which fails in monorepos where `bun` or `pnpm` hoists dependencies to the root. Adjusting this path to `../node_modules/heroui-native/lib` is necessary to ensure components like `Surface` and `Button` retain their styles in production builds.¹²

Backend Logic and API Type Safety

The inclusion of a standalone Express backend running on Node.js distinguishes this stack from pure Next.js implementations. This architectural choice is often driven by the need for long-running processes, specialized middleware, or integration with legacy systems that require a persistent server environment rather than transient edge functions.¹³

tRPC v11 and Type-Safe Communication

tRPC bridges the gap between the Express server and the frontends by enabling end-to-end type safety without the need for code generation. The Express server defines an `appRouter` composed of various procedures (queries and mutations). By exporting only the `type` of this router, the frontend applications can infer the exact input and output shapes of every API call.⁴

The integration process involves:

1. **Server Definition:** Initializing tRPC on the server and defining procedures using a schema validator like `Zod`.⁴
2. **Context Creation:** Defining a `createContext` function that extracts user sessions (via `Better-Auth`) and database clients for use within procedures.¹⁷

3. **Router Export:** Exporting the AppRouter type from the server package.¹⁶
4. **Client Consumption:** Using @trpc/react-query or @trpc/client in the Next.js and React Native apps to interact with the API with full TypeScript autocomplete.⁴

This mechanism effectively removes the "imaginary boundary" between frontend and backend, as a change in the server's return type will immediately trigger a build error in the client applications.⁴

Identity and Session Management: Better-Auth

Authentication in a multi-runtime environment (Next.js, React Native, and Express) requires a unified session management system. Better-Auth is selected for its framework-agnostic nature and its ability to handle both server-side and client-side authentication flows with a consistent API.¹³

Cross-Runtime Session Persistence

The configuration of Better-Auth in this stack involves mounting the handler in the Express server while using Next.js rewrites to ensure the browser perceives the auth API as being on the same domain. This is crucial for cookie-based session persistence.¹⁰

Framework/Context	Handler Helper	implementation
Express Backend	toNodeHandler(auth)	Mounted at /api/auth/*. ¹³
Next.js Frontend	toNextJsHandler(auth)	Used in API routes if direct auth is needed. ⁹
React Native Client	createAuthClient	Interacts with the backend via HTTP. ¹⁸

A notable security and implementation detail arises when calling authenticated endpoints from Next.js Server Components. Since cookies are not automatically forwarded in server-to-server requests, developers must manually extract cookies from the next/headers and attach them to the fetch options of the Better-Auth client. Alternatively, a proxy plugin can be added to the client to automate this injection.¹⁰

Persistence Layer: Turso and Drizzle ORM

The data layer utilizes SQLite as the engine, Turso as the managed provider, and Drizzle as the ORM. This combination is optimized for low-latency operations and "edge-ready" deployments.¹⁹

libSQL and Embedded Replicas

Turso is built on libSQL, which extends SQLite with server-client capabilities and global replication.¹⁹ One of the "killer features" of Turso in this configuration is the use of embedded replicas. This allows the application to maintain a local SQLite file that stays in sync with the primary database. Reads are performed against the local file with near-zero latency, while writes are synchronized to the cloud.²⁰

The Drizzle ORM provides a type-safe interface for this database. Unlike heavier ORMs, Drizzle acts as a thin wrapper over SQL, allowing for granular control and excellent performance in Node.js environments.¹⁵ The migration workflow is handled through drizzle-kit, which generates SQL migration files from TypeScript schema definitions. These files are then applied to the Turso database using the libsql driver.¹⁹

Migration and Sync Workflow

In a development environment, the workflow typically follows a structured sequence to ensure the local and remote schemas remain aligned.⁵

1. **Schema Definition:** Tables and relationships are defined in packages/db/schema.ts.¹⁹
2. **Migration Generation:** `bun run db:generate` creates a new SQL migration in the migrations/ folder.⁵
3. **Migration Application:** `bun run db:migrate` applies the SQL changes to the Turso cloud instance.²²
4. **Local Sync:** For local-first or replicated scenarios, the `sync()` method is called to pull remote changes into the local embedded replica.²¹

Financial Infrastructure: Polar Merchant of Record

For monetization, the stack utilizes Polar instead of a standard Stripe implementation. The fundamental difference lies in the Merchant of Record (MoR) model. While Stripe acts as a payment processor, leaving tax compliance and liability to the developer, Polar assumes the MoR role, handling global tax collection, remittance, and filing.³

Comparison of Financial Models

The choice of Polar is particularly beneficial for indie developers and small teams who wish to sell software globally without navigating the complexities of international VAT and GST laws.³

Metric	Polar (MoR)	Stripe (Processor)

Base Fee	4% + 40¢	2.9% + 30¢
International Card Fee	+1.5%	+1.5% (approx)
Subscription Fee	Included	+0.5% - 0.7%
Tax Compliance	Included (Global)	Developer responsibility
Liability	Assumed by Polar	Assumed by Developer

Polar is built on top of Stripe's "rails," ensuring high reliability while providing an opinionated layer for software-specific features like tiered memberships, gated GitHub repositories, and digital product hosting.³ Integration is simplified through the `@polar-sh/nextjs` and `@polar-sh/sdk` packages, allowing developers to set up a full checkout and webhook lifecycle in a few lines of code.²⁵

Webhook Integration for Provisioning

Webhooks are the primary mechanism for keeping the local database in sync with the payment state. Polar sends events for `order.created`, `subscription.updated`, and `customer.state_changed`.²⁵ The webhook handler, typically located at `/api/webhook/polar` in the Next.js or Express app, verifies the `POLAR_WEBHOOK_SECRET` before updating the user's "benefit" status in the Drizzle-managed SQLite database.²⁵

Observability and Monitoring: Better Stack

To ensure production stability, the stack integrates with the Better Stack ecosystem. This provides a unified platform for uptime monitoring, log management, and infrastructure observability.²⁷

Telemetry and Collector Integration

The Better Stack collector is the recommended method for gathering telemetry data. It leverages eBPF to instrument clusters (like Kubernetes or Docker) without requiring extensive code changes.²⁸

- **Log Management:** The collector scrapes Nginx and Apache metrics and processes Docker logs, attaching container labels as metadata.²⁸
- **OpenTelemetry:** All services are configured to send data via OpenTelemetry SDKs, which Better Stack uses to correlate logs and traces.²⁸
- **Infrastructure Monitoring:** The collector monitors host-level metrics and provides an eBPF-based service map to visualize dependencies.²⁸

This observability layer is critical for debugging the distributed nature of the stack, where a request may pass through a Next.js proxy, an Express backend, a tRPC router, and finally a Turso database.¹⁰

Progressive Web App (PWA) and Mobile Enhancement

The PWA addon enables the web frontend to function with app-like characteristics, including offline support and home-screen installability.³⁰ This is achieved through the generation of a manifest.json file and the registration of service workers.

PWA Characteristics and Benefits

Feature	implementation	Value Proposition
Installability	Web Manifest + Link tag	Bypasses app stores for rapid user adoption. ³⁰
Offline Capability	Service Workers + Caching	Enhances usability in low-network conditions. ³¹
Background Sync	Service Worker Sync API	Ensures data consistency for deferred actions. ³⁰
Push Notifications	Push API	Increases user engagement and retention. ³⁰

The integration of PWA features in a Next.js environment is facilitated by libraries like next-pwa, which uses Workbox to manage the service worker lifecycle and caching strategies.³² This complements the mobile-first approach of the React Native component, providing multiple touchpoints for users across different devices.⁸

Theoretical Latency and Performance Modeling

In a distributed stack using SQLite at the edge, performance is heavily influenced by the proximity of the application runtime to the data replica. The total latency (T_{total}) for a

request can be modeled as the sum of network transit time, application processing, and database execution.²⁰

$$T_{total} = T_{network} + T_{app} + T_{db}$$

Where T_{db} in a Turso embedded replica environment is nearly instantaneous ($< 1ms$) for read operations, as the data is served from the local filesystem. For write operations, the latency increases due to the need for synchronization with the primary Turso instance:

$$T_{db_write} = T_{process} + T_{cloud_sync}$$

By deploying the Express server and Next.js backend in regions that match the Turso replicas, the $T_{network}$ component is minimized, resulting in a stack that can deliver sub-100ms response times globally.¹⁹

Future Outlook and Scalability Insights

The Better-T-Stack configuration chosen—Express on Node.js with Next.js and React Native—positions the project for high architectural flexibility. While serverless options are popular, the choice of a persistent Node.js runtime for the backend allows for the use of advanced features like tRPC subscriptions via WebSockets, which are often limited in serverless environments.⁴

The modular nature of the monorepo ensures that individual components can be swapped or upgraded with minimal friction. For instance, if the project outgrows SQLite, the Drizzle ORM allows for a relatively straightforward migration to PostgreSQL (such as Neon or Supabase) by updating the dialect and connection string in the packages/db configuration.¹ Similarly, the use of Polar as an MoR provides a "limitless" scaling path for international commerce, as the administrative burden of tax compliance does not grow with the number of jurisdictions served.³

Conclusion on Integrated System Design

The synthesis of these technologies into the Better-T-Stack creates a cohesive, type-safe environment that addresses the major pain points of modern software development: configuration fatigue, type mismatches, authentication complexity, and global tax compliance. The combination of pnpm and Turborepo provides the necessary structure to manage this complexity, while tRPC and Better-Auth ensure that the security and communication layers remain robust across multiple runtimes. By leveraging Turso and Polar, the stack optimizes both the technical performance of the application and the operational efficiency of the business, creating a foundation that is as viable for a solo developer as it is for a scaling engineering

team.¹

Works cited

1. create-better-t-stack - NPM, accessed May 12, 2026, <https://www.npmjs.com/package/create-better-t-stack>
2. Better-T-Stack - Best of JS, accessed May 12, 2026, <https://bestofjs.org/projects/create-better-t-stack>
3. Polar vs Stripe, accessed May 12, 2026, <https://polar.sh/resources/comparison/stripe>
4. Setting Up tRPC with Next.js App Router - Nisab Mohd, accessed May 12, 2026, <https://nisabmohd.vercel.app/trpc-app-router>
5. AmanVarshney01/Better-T-Stack: Tanstack Router, Hono, tRPC, Better Auth, Drizzle, Tailwind Template - GitHub, accessed May 12, 2026, <https://github.com/AmanVarshney01/Better-T-Stack>
6. Building a full-stack TypeScript application with Turborepo - LogRocket Blog, accessed May 12, 2026, <https://blog.logrocket.com/build-full-stack-typescript-application-turborepo/>
7. create-better-t-stack/AGENTS.md at main - GitHub, accessed May 12, 2026, <https://github.com/AmanVarshney01/create-better-t-stack/blob/main/AGENTS.md>
8. NoQuarterTeam/boilerplate: Tanstack Start + Expo - GitHub, accessed May 12, 2026, <https://github.com/NoQuarterTeam/boilerplate>
9. Next.js integration - Better Auth, accessed May 12, 2026, <https://better-auth.com/docs/integrations/next>
10. Using better-auth on Nextjs with existing backend (not within Nextjs api) #5336 - GitHub, accessed May 12, 2026, <https://github.com/better-auth/better-auth/discussions/5336>
11. Uniwind Template for React Native and Expo — 40+ Components ..., accessed May 12, 2026, https://medium.com/@the.zeman_97093/uniwind-template-for-react-native-and-expo-40-components-dark-and-light-mode-3f7e2d610b8a
12. Expo + Uniwind in Monorepo uses wrong import path for HeroUI styles · Issue #810 - GitHub, accessed May 12, 2026, <https://github.com/AmanVarshney01/create-better-t-stack/issues/810>
13. Express Integration | Better Auth, accessed May 12, 2026, <https://better-auth.com/docs/integrations/express>
14. How to Use TypeScript for Full-Stack Web Development - MindStudio, accessed May 12, 2026, <https://www.mindstudio.ai/blog/how-to-use-typescript-full-stack-development>
15. Create a Full-Stack app with tRPC, Express and React | by Alan - JavaScript in Plain English, accessed May 12, 2026, <https://javascript.plainenglish.io/create-a-full-stack-app-with-trpc-react-and-express-35ed95f0851a>
16. Building a tRPC Router from First Principles - DEV Community, accessed May 12,

2026,

<https://dev.to/gabrielanhaia/building-a-trpc-router-from-first-principles-2o6a>

17. tRPC Discord Server, accessed May 12, 2026,
https://discord-questions.trpc.io/?cursor=B8JiTz8oLLK_7iLA8Cz-qJLTeQZlhF62KD9_I9qgRiqS5Hx9EtiUJudS-cPqmu2gcRdY97OWQT5mb6YMLrH9ZF7afPNqoSiv5YQBdpXOCurr0fCQlxUDukfP8NcYxsywo9fxd6ImOiR4rPud21QjDxLgC3oMG9Oa_QS2ybMJZHBxIzM
18. Installation | Better Auth, accessed May 12, 2026,
<https://www.better-auth.com/docs/installation>
19. Drizzle with Turso, accessed May 12, 2026,
<https://orm.drizzle.team/docs/tutorials/drizzle-with-turso>
20. Turso Database Guide 2026: Edge Revolution | Codebrand Blog, accessed May 12, 2026, <https://www.codebrand.us/blog/turso-database-complete-guide-2026/>
21. Building a Local-First Tauri App with Drizzle ORM, Encryption, and Turso Sync (tauri-plugin-libsql) - DEV Community, accessed May 12, 2026,
<https://dev.to/luakun/building-a-local-first-tauri-app-with-drizzle-orm-encryption-and-turso-sync-31pn>
22. Drizzle + Turso, accessed May 12, 2026, <https://docs.turso.tech/sdk/ts/orm/drizzle>
23. Polar: a better Stripe alternative | Raul Carini, accessed May 12, 2026,
<https://www.raulcarini.dev/blog/polar-better-stripe-alternative>
24. Why Polar is the best way to monetize your software, accessed May 12, 2026,
<https://polar.sh/resources/why>
25. Integrate Polar with Next.js - Polar, accessed May 12, 2026,
<https://polar.sh/docs/guides/nextjs>
26. Vite + Clerk + Convex + Polar - Tempo, accessed May 12, 2026,
<https://tempolabsinc.mintlify.app/ViteClerkConvexPolar>
27. Getting started | Better Stack Documentation, accessed May 12, 2026,
<https://betterstack.com/docs/getting-started/welcome/>
28. Better Stack collector | Better Stack Documentation, accessed May 12, 2026,
<https://betterstack.com/docs/logs/collector/>
29. Better Stack - GitHub, accessed May 12, 2026, <https://github.com/BetterStackHQ>
30. Progressive Web App (PWA) Development Ultimate Guide - Riseup Labs, accessed May 12, 2026,
<https://riseuplabs.com/pwa-development-ultimate-guide/>
31. A deep-dive on a Progressive Web App implementation for a React-based App Platform (DHIS2), accessed May 12, 2026,
<https://developers.dhis2.org/blog/2023/08/pwa-tech-1/>
32. tribixbite/awesome - GitHub, accessed May 12, 2026,
<https://github.com/tribixbite/awesome>